

Aula 5 - Shell Avançado I: Canais de Comunicação, Redirecionamento, Pipes, Processos e Threads

1. Redirecionamento e Pipes

Nos capítulos anteriores, você aprendeu comandos como **ls**, **cat**, **grep**, **find**, **date**, **curingas** para facilitar os comandos. Todavia, cada um faz uma tarefa específica e joga o resultado na tela. Mas no dia a dia, raramente queremos apenas *ver* o resultado, queremos **salvá-lo** em um arquivo, **filtrar** apenas a parte relevante, ou **alimentar** outro comando com aquele dado. No Linux, programas se comunicam como peças em uma linha de montagem, passando dados uns para os outros. As duas mecânicas que viabilizam isso são o **redirecionamento** (desviar dados para arquivos) e os **pipes** (desviar dados para outros programas). Todavia, antes é necessário compreender como funcionam os canais de comunicação dentro do sistema linux.

1.1 Os Três Canais de Comunicação

Quando um programa é inicializado, o sistema operacional automaticamente conecta três canais de comunicação a ele. Esses canais existem para **todo** programa, sem exceção, desde o **ls** mais simples até um servidor web complexo. Pense neles como três canos de encanamento: o primeiro traz dados para dentro do programa (a entrada), o segundo leva os resultados normais para fora (a saída), e o terceiro, completamente separado do segundo, leva apenas mensagens de erro para fora.

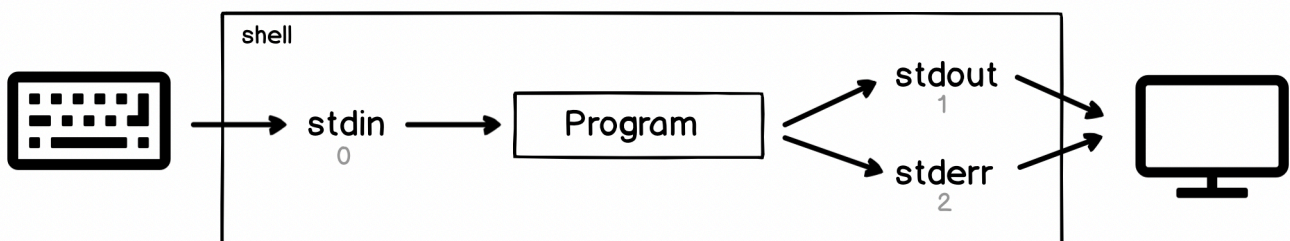


Figura 1: Os três canais padrão de comunicação de todo programa no Linux.

Observe na Figura 1 como o fluxo funciona. O teclado envia dados para o programa através do canal **stdin** (número 0). O programa processa esses dados e produz dois tipos de saída por caminhos separados: os resultados normais saem pelo **stdout** (número 1) e as mensagens de erro saem pelo **stderr** (número 2). Ambos os canais de saída apontam para a tela do monitor, e é por isso que, a olho nu, tudo parece uma coisa só.

Canal	Nome Técnico	Nº	Direção	Descrição
stdin	Standard Input (Entrada Padrão)	0	Teclado → Programa	O canal por onde o programa recebe dados de entrada
stdout	Standard Output (Saída Padrão)	1	Programa → Tela	O canal por onde saem os resultados normais da execução
stderr	Standard Error (Saída de Erro)	2	Programa → Tela	Um canal separado, exclusivo para mensagens de erro

Na prática, quando você executa `ls /bin`, o programa recebe o argumento pelo stdin (canal 0), processa a listagem e envia o resultado pelo stdout (canal 1), que aparece na tela. Se você digitar `ls /pasta_que_nao_existe`, a mensagem de erro será enviada pelo stderr (canal 2), que também aparece na tela, mas por um caminho completamente diferente.

A existência de dois canos de saída distintos é uma decisão de design fundamental do Unix, pois permite tratar resultados e erros de forma independente. É como se o programa tivesse duas torneiras jogando água na mesma pia: por dentro são dois encanamentos independentes, mesmo que na pia pareçam um só.

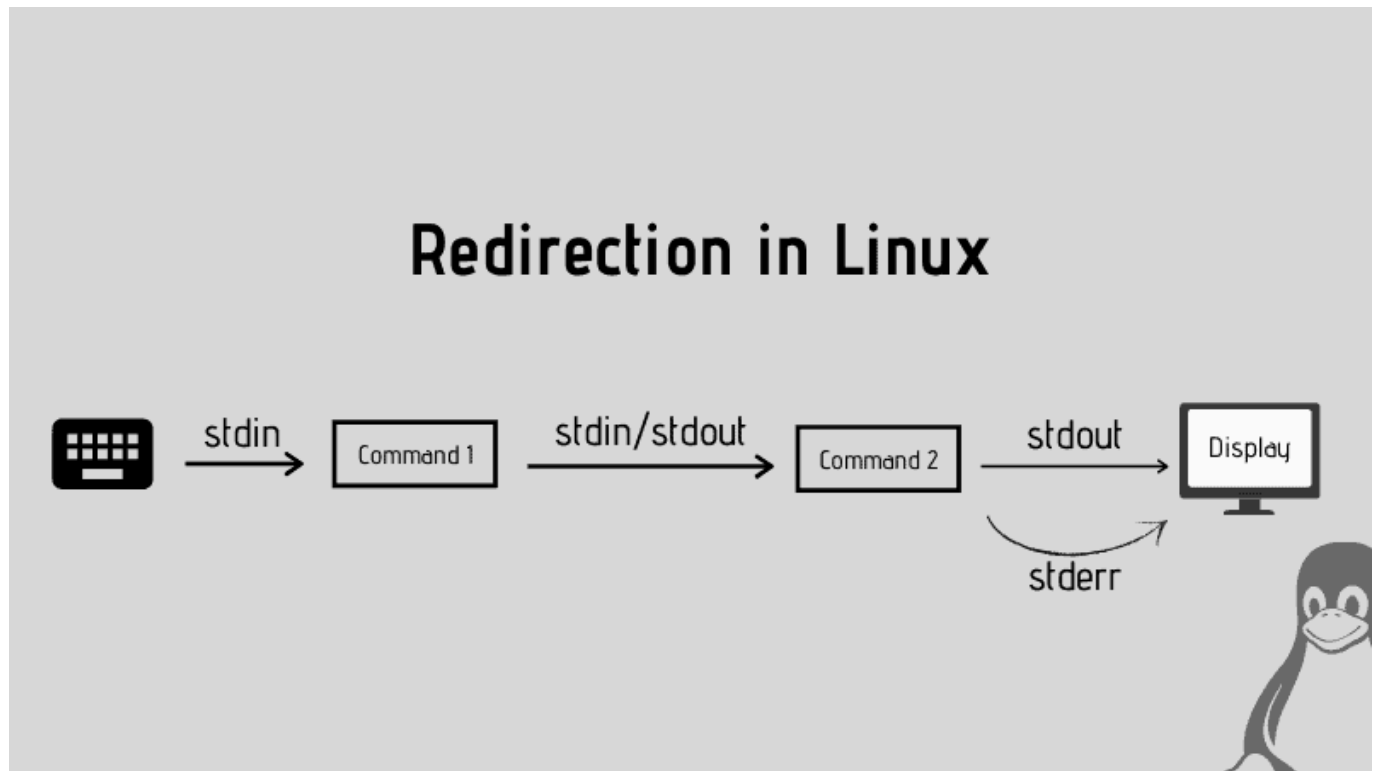


Figura 2: *stdin*, *stdout* e *stderr*.

A Figura 2 mostra o panorama completo. Quando conectamos dois comandos em sequência, a saída padrão (stdout) do primeiro se transforma na entrada padrão (stdin) do segundo, formando uma corrente. Enquanto isso, o stderr mantém seu caminho independente, indo direto para a tela.

Agora que entendemos por onde os dados fluem, podemos aprender as duas formas de reconectar esses canos. O **redirecionamento** reconecta os canais a **arquivos**, permitindo salvar resultados ou capturar erros em disco. Os **pipes** reconectam o stdout de um comando diretamente ao stdin de **outro comando**, permitindo encadear operações sem criar arquivos intermediários.

Para praticar esses conceitos com exemplos reais, vamos retomar o projeto **Sonho/W2G** que construímos no Capítulo 4. Navegue até a sua pasta **Documentos** e recrie a estrutura exatamente como apresentada na árvore abaixo, incluindo os subdiretórios e os arquivos vazios:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ tree
.
├── configs
│   ├── roboA.cfg
│   ├── roboB.cfg
│   ├── roboC.cfg
│   └── roboZ.cfg
```

```
├── docs
│   ├── manual_v1.pdf
│   ├── manual_v2.pdf
│   └── relatorio_erro.pdf
├── logs
│   ├── erro_critico.log
│   ├── erro_motor.txt
│   ├── sensor_frontal.log
│   └── sensor_traseiro.log
├── scripts
│   ├── main.py
│   ├── script2.7.py
│   ├── script3.10.py
│   └── script3.8.py
└── testes
    ├── teste1.py
    ├── teste10.py
    └── teste2.py
```

Com o ambiente pronto, vamos começar a reconectar os canos.

1.2 Redirecionamento

O redirecionamento permite desviar os canais de comunicação para arquivos em vez dos destinos padrão. Os resultados que iriam para a tela passam a ser gravados em um arquivo, e os dados que viriam do teclado passam a ser lidos de um arquivo. É a ferramenta essencial para gerar relatórios, salvar logs e automatizar qualquer tarefa que precise de registros persistentes.

1.2.1 Redirecionando a Saída com **>** (Sobrescrever)

O operador **>** é o mais básico e o mais usado. Ele pega o canal stdout de um comando e desvia para um arquivo. Se o arquivo de destino não existir, o sistema o cria automaticamente. Se já existir, o conteúdo anterior é **apagado e substituído** pelo novo.

```
# Sem redirecionamento, o resultado aparece na tela
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ date
qua 19 mar 2025 10:15:32 -03

# Com >, a tela fica silenciosa, pois a saída foi desviada para o arquivo
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ date > data_atual.txt
```

Perceba que ao usar o **>**, nada apareceu na tela. Isso é esperado: o stdout foi reconectado ao arquivo e não sobrou nada para o terminal exibir. Para conferir o que foi gravado, usamos o **cat**:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat data_atual.txt
qua 19 mar 2025 10:15:32 -03
```

Um cuidado importante: se executarmos o mesmo comando novamente, o arquivo é **sobrescrito por completo**. O conteúdo anterior desaparece sem aviso, substituído pelo novo resultado:

```
# Executamos de novo alguns segundos depois
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ date > data_atual.txt
```

A data anterior foi substituída pela nova:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat data_atual.txt
qua 19 mar 2025 10:15:35 -03
```

Isso funciona com **qualquer** comando que produza saída na tela. Podemos salvar a listagem de uma pasta, o resultado de uma busca com **grep**, ou até mesmo escrever texto puro com **echo** (**spoiler alert**). Vamos a mais alguns exemplos dentro do nosso projeto.

Para salvar a lista de todos os scripts Python em um arquivo de inventário, combinamos o **ls** com o curinga ***.py** que aprendemos na aula anterior:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls scripts/*.py >
lista_scripts.txt
```

Conferindo o resultado:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat lista_scripts.txt
scripts/main.py
scripts/script2.7.py
scripts/script3.10.py
scripts/script3.8.py
```

Agora um exemplo que combina o **grep** com o **>** para filtrar conteúdo. Lembra do arquivo **logs/erro_critico.log** que criamos vazio no Capítulo 4 usando o **touch**? Vamos preenchê-lo com dados para que possamos trabalhar com ele. Abra o arquivo com o **nano** e adicione o seguinte conteúdo:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ nano
logs/erro_critico.log
```

```
erro 1
erro 2
Erro 3
```

```
ERRO 4
eRRO 5
```

Salve com **Ctrl+O** e saia com **Ctrl+X**. Agora vamos usar o **grep** para buscar a palavra **erro** dentro desse arquivo e salvar apenas as linhas encontradas em um novo arquivo:

```
(base)      thallys@thallys-note:~/Documentos/Sonho/W2G$      grep      "erro"
logs/erro_critico.log > erros_encontrados.txt
```

Conferindo o resultado:

```
(base)      thallys@thallys-note:~/Documentos/Sonho/W2G$      cat
erros_encontrados.txt
erro 1
erro 2
```

Observe que das cinco linhas originais, apenas duas foram capturadas. Isso acontece porque o **grep**, por padrão, diferencia maiúsculas de minúsculas. A busca por **"erro"** encontrou apenas as linhas escritas exatamente assim, com todas as letras minúsculas. As linhas com **Erro**, **ERRO** e **eRRO** foram ignoradas por terem capitalização diferente. Essa sensibilidade é uma característica importante do Linux que já vimos desde o Capítulo 2: o sistema trata **a** e **A** como caracteres completamente distintos.

Caso precise ignorar essa diferença, o **grep** aceita a flag **-i** (case-insensitive). Combinando essa flag com o redirecionamento, conseguimos capturar todas as variações de uma só vez:

```
(base)      thallys@thallys-note:~/Documentos/Sonho/W2G$      grep      -i      "erro"
logs/erro_critico.log > erros_encontrados.txt
```

Conferindo o resultado:

```
(base)      thallys@thallys-note:~/Documentos/Sonho/W2G$      cat
erros_encontrados.txt
erro 1
erro 2
Erro 3
ERRO 4
eRRO 5
```

Agora todas as cinco linhas foram capturadas, independentemente de como a palavra estava escrita. Esse é um padrão muito comum no dia a dia: usar **grep -i** para garantir que nenhuma variação escape da busca, e o **>** para salvar o resultado filtrado em um arquivo de análise.

Até aqui, porém, temos um problema. Toda vez que usamos o `>`, o conteúdo anterior do arquivo é destruído. Se quisermos rodar esse filtro novamente amanhã e **acumular** os resultados em vez de substituí-los, precisamos de outro operador. É exatamente isso que o `>>` resolve.

1.2.2 Redirecionando a Saída com `>>` (Adicionar)

Até agora, toda vez que usamos o `>`, o conteúdo anterior do arquivo era destruído. Mas em muitas situações, queremos **acumular** informações ao longo do tempo, como um diário de bordo onde cada nova entrada é escrita na página seguinte sem arrancar as anteriores. Para isso, usamos o operador `>>`, que funciona exatamente como o `>`, mas em vez de apagar o conteúdo existente, ele **adiciona ao final** do arquivo.

Antes de demonstrar o `>>` na prática, precisamos conhecer um comando novo que será nosso companheiro constante daqui em diante: o `echo`.

O comando `echo`

O `echo` é o equivalente do `print()` no Python. Sua função é simples e direta: ele imprime na tela o texto que você passar como argumento.

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo 'Olá, mundo!'
Olá, mundo!

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo 'O usuário logado
é: thallys'
O usuário logado é: thallys
```

Ele também aceita a flag `-e`, que ativa caracteres especiais como `\n` (quebra de linha) e `\t` (tabulação):

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo -e 'Linha 1\nLinha
2\nLinha 3'
Linha 1
Linha 2
Linha 3

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo -e
'Nome:\tThallys\nIdade:\t25'
Nome:  Thallys
Idade: 25
```

Sozinho, o `echo` parece trivial, pois imprimir texto na tela não é muito útil quando podemos simplesmente lê-lo. Porém, combinado com redirecionamento, ele se transforma em uma ferramenta poderosa para **criar e editar arquivos sem abrir nenhum editor de texto**. É exatamente assim que vamos usá-lo agora.

Combinando `echo` com `>` e `>>`

Vamos montar um relatório do sistema passo a passo, usando `echo` para escrever os textos fixos, `date` para a data e `ls` para a listagem. O primeiro comando usa `>` para criar o arquivo do zero, e todos os seguintes

usam >> para ir adicionando sem apagar o que já está lá:

```
# 0 > cria o arquivo e escreve o cabeçalho (se já existisse, seria sobrescrito)
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo '=== RELATÓRIO DO SISTEMA ===' > relatorio.txt
```

A partir daqui, tudo com >> para adicionar ao final:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo "" >> relatorio.txt
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo 'Data da análise:' >> relatorio.txt
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ date >> relatorio.txt
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo "" >> relatorio.txt
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ echo 'Arquivos no diretório:' >> relatorio.txt
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls -lh >> relatorio.txt
```

Cada comando adicionou sua contribuição ao final do arquivo, sem interferir no que já havia sido escrito. Conferindo o resultado acumulado:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat relatorio.txt
===RELATÓRIO DO SISTEMA===
```

Data da análise:

Arquivos no diretório:

```
total 48K
drwxrwxr-x 2 thallys thallys 4,0K mar 18 18:35 arquivos
drwxrwxr-x 2 thallys thallys 4,0K mar 18 20:41 configs
-rw-rw-r-- 1 thallys thallys 29 mar 24 10:00 data_atual.txt
drwxrwxr-x 2 thallys thallys 4,0K mar 18 18:37 delivery
drwxrwxr-x 2 thallys thallys 4,0K mar 18 16:33 docs
-rw-rw-r-- 1 thallys thallys 35 mar 24 10:16 erros_encontrados.txt
-rw-rw-r-- 1 thallys thallys 116 mar 24 10:01 lista_scripts.txt
drwxrwxr-x 2 thallys thallys 4,0K mar 24 10:03 logs
-rw-rw-r-- 1 thallys thallys 72 mar 24 10:39 relatorio.txt
drwxrwxr-x 2 thallys thallys 4,0K mar 18 18:35 revisao
drwxr-xr-- 2 thallys thallys 4,0K mar 18 20:48 scripts
drwxrwxr-x 7 thallys thallys 4,0K mar 18 18:44 testes
```

Resumindo

Operador	Comportamento	Quando Usar
>	Cria ou sobrescreve o arquivo	Quando quer um resultado limpo e novo
>>	Cria ou adiciona ao final	Quando quer acumular registros, como logs e relatórios

1.2.3 Redirecionando a Saída de Erro com 2>

Até agora, todos os nossos exemplos produziram apenas resultados válidos. Mas o que acontece quando um comando encontra um problema? Vamos provocar um erro de propósito tentando listar uma pasta que não existe dentro do nosso projeto:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls sensores/
ls: não foi possível acessar 'sensores/': Arquivo ou diretório inexistente
```

Essa mensagem de erro foi enviada pelo canal **stderr** (canal 2), não pelo stdout. Parece um detalhe irrelevante, já que ambos aparecem na tela da mesma forma. Mas observe o que acontece quando tentamos redirecionar essa saída com o >:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls sensores/ >
resultado.txt
ls: não foi possível acessar 'sensores/': Arquivo ou diretório inexistente
```

A mensagem de erro **continuou aparecendo na tela**, mesmo com o > ativo. Isso acontece porque o > redireciona apenas o canal 1 (stdout). Como o erro viajou pelo canal 2 (stderr), ele passou direto sem ser capturado. Para interceptar esse outro cano, precisamos usar o operador 2>, que redireciona explicitamente o canal de erro:

```
# 0 2> captura o stderr e desvia para o arquivo
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls sensores/ 2>
erros.txt

# Agora a tela ficou silenciosa, o erro foi capturado
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat erros.txt
ls: não foi possível acessar 'sensores/': Arquivo ou diretório inexistente
```

Esse recurso se torna especialmente útil quando um comando produz resultados válidos e erros ao mesmo tempo. Imagine que queremos listar o conteúdo das pastas **configs** e **sensores**. A primeira existe no nosso projeto, a segunda não. Podemos separar cada canal para um arquivo diferente:

```
# stdout vai para resultados.txt, stderr vai para erros.txt
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls configs/ sensores/ >
resultados.txt 2> erros.txt
```

Conferindo cada arquivo separadamente:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat resultados.txt
configs/:
roboA.cfg
roboB.cfg
roboC.cfg
roboZ.cfg

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat erros.txt
ls: não foi possível acessar 'sensores/': Arquivo ou diretório inexistente
```

Perceba como os dois canais foram separados cirurgicamente. Os resultados válidos da pasta `configs` foram para um arquivo, e a mensagem de erro sobre a pasta `sensores` foi para outro. Sem o `2>`, o erro teria aparecido misturado na tela enquanto apenas os resultados seriam salvos, e perderíamos o registro de que algo deu errado.

1.2.4 Combinando stdout e stderr com `&>`

No exemplo anterior, separamos resultados e erros em arquivos diferentes usando `>` e `2>` ao mesmo tempo. Porém, nem sempre essa separação é necessária. Em muitas situações, como quando estamos depurando um problema ou gerando um log completo de uma execução, queremos simplesmente gravar **tudo** em um único arquivo, sem nos preocupar em distinguir o que é resultado e o que é erro. Para isso, usamos o operador `&>`, que junta os dois canais em um só destino.

Usando o mesmo exemplo das pastas `configs` e `sensores`:

```
# 0 &> captura stdout e stderr juntos no mesmo arquivo
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls configs/ sensores/
&> log_completo.txt
```

Conferindo o resultado:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat log_completo.txt
ls: não foi possível acessar 'sensores/': Arquivo ou diretório inexistente
configs/:
roboA.cfg
roboB.cfg
roboC.cfg
roboZ.cfg
```

Perceba que tanto a mensagem de erro sobre `sensores/` quanto a listagem válida de `configs/` foram gravadas no mesmo arquivo. Esse é o comportamento ideal para logs de execução, onde queremos um registro completo de tudo que aconteceu, incluindo os problemas.

Assim como o `>` tem sua versão de adição `>>`, o `&>` também possui o `&>>`, que acumula as saídas no final do arquivo em vez de sobrescrever. Essa combinação é particularmente útil em scripts que rodam periodicamente e precisam manter um histórico contínuo de execuções. Vamos simular duas execuções seguidas para observar o acúmulo.

A primeira execução cria o arquivo com `&>`:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls configs/ sensores/ &> log_historico.txt
```

Conferindo, temos o registro da primeira execução com o erro e a listagem juntos:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat log_historico.txt
ls: não foi possível acessar 'sensores/': Arquivo ou diretório inexistente
configs/:
roboA.cfg
roboB.cfg
roboC.cfg
roboZ.cfg
```

Agora executamos um segundo comando, desta vez com `&>>` para adicionar ao final sem destruir o conteúdo anterior:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls docs/ testes/ &>> log_historico.txt
```

Conferindo novamente, o arquivo agora contém o resultado das **duas** execuções acumuladas:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat log_historico.txt
ls: não foi possível acessar 'sensores/': Arquivo ou diretório inexistente
configs/:
roboA.cfg
roboB.cfg
roboC.cfg
roboZ.cfg
docs/:
manual_v1.pdf
manual_v2.pdf
relatorio_erro.pdf
testes/:
fase_1
fase_2
fase_3
fase_4
```

```
fase_5
teste10.py
```

O conteúdo da primeira execução permaneceu intacto, e o resultado da segunda foi adicionado logo abaixo.

1.2.5 Redirecionando a Entrada com <

Todos os operadores que vimos até agora manipulam a **saída** dos comandos, desviando resultados e erros para arquivos. O operador < faz o caminho inverso: ele alimenta a **entrada** de um comando com o conteúdo de um arquivo, como se você estivesse digitando aquele conteúdo pelo teclado, linha por linha.

Na prática, o < é menos usado que o >, pois a maioria dos comandos já aceita o nome do arquivo como argumento direto (como `cat logs/erro_critico.log`). Porém, existem situações em que ele faz diferença real, especialmente quando combinamos entrada e saída no mesmo comando ou quando precisamos de uma saída limpa para processamento posterior.

Para explorar essas situações, vamos usar dois comandos novos: o `sort` e o `wc`.

O comando `sort`

O `sort` ordena as linhas de um texto em ordem alfabética. Sua estrutura é simples: basta passar o arquivo como argumento e ele reorganiza as linhas. Antes de testá-lo, vamos atualizar o conteúdo do nosso arquivo de log para ter dados mais variados. Abra o arquivo com o `nano` e substitua o conteúdo pelo seguinte:

```
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$          nano
logs/erro_critico.log
```

```
eRro 1
eRro 2
Erro 3
Bom demais!
erro 1
erro 2
Erro 3
ERRO 4
eRRO 5
```

Salve com `Ctrl+O` e saia com `Ctrl+X`. Agora vamos ver como o `sort` reorganiza essas linhas:

```
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$          sort
logs/erro_critico.log
Bom demais!
erro 1
eRro 1
erro 2
```

```
eRro 2
Erro 3
Erro 3
ERRO 4
eRRO 5
```

Observe que a linha **Bom demais!** subiu para o topo porque o **B** maiúsculo vem antes das letras minúsculas na tabela de caracteres que o sistema usa para comparação. As linhas que começam com **e** minúsculo e **E** maiúsculo foram intercaladas seguindo essa mesma lógica. O **sort** não interpreta o significado das palavras, ele simplesmente compara os caracteres posição por posição, da esquerda para a direita.

Ele também aceita flags que mudam o tipo de ordenação. A flag **-r** inverte a ordem, exibindo do último para o primeiro, e a flag **-n** ordena numericamente, para que o 2 venha antes do 10 em vez de depois, como aconteceria na ordenação alfabética padrão. Veremos essas variações com mais detalhes na seção de Pipes.

O comando **wc**

O **wc** (abreviação de **word count**) é uma ferramenta de contagem que analisa o conteúdo de um arquivo e retorna estatísticas sobre ele. Quando executado sem nenhuma flag, ele exibe três números de uma vez: a quantidade de linhas, de palavras e de bytes do arquivo.

```
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$          wc
logs/erro_critico.log
10 18 70 logs/erro_critico.log
```

A saída nos diz que o arquivo possui 9 linhas, 17 palavras e 83 bytes. Na maioria das situações, porém, precisamos de apenas uma dessas informações. Para isso, usamos flags específicas. A flag **-l** retorna apenas a contagem de linhas, que é de longe a mais utilizada no dia a dia:

```
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$          wc          -l
logs/erro_critico.log
10 logs/erro_critico.log
```

Já a flag **-w** retorna apenas a contagem de palavras:

```
(base)          thallys@thallys-note:~/Documentos/Sonho/W2G$          wc          -w
logs/erro_critico.log
18 logs/erro_critico.log
```

Perceba que em todos os exemplos o **wc** exibe o nome do arquivo ao lado do número. Esse detalhe será importante para entender a diferença que o operador **<** faz, como veremos a seguir.

Agora que conhecemos os dois comandos, vamos entender onde o **<** entra em cena.

O **<** na prática: alimentando comandos pela entrada padrão

Tanto o `sort` quanto o `wc` aceitam o arquivo como argumento direto, conforme vimos acima. Mas eles também podem receber os dados pela entrada padrão usando o `<`. Com o `sort`, o resultado é literalmente idêntico nas duas formas, sem nenhuma diferença visível. Então por que o `<` existe?

A resposta aparece com o `wc`. Observe como a saída muda dependendo da forma que o arquivo é fornecido:

```
(base)      thallys@thallys-note:~/Documentos/Sonho/W2G$      wc      -l
logs/erro_critico.log
10 logs/erro_critico.log

(base)      thallys@thallys-note:~/Documentos/Sonho/W2G$      wc      -l      <
logs/erro_critico.log
10
```

Na primeira forma, o `wc` recebeu o arquivo como argumento, sabe o nome dele e o exibe na saída. Na segunda forma, o `<` alimentou o conteúdo pela entrada padrão, e do ponto de vista do `wc`, alguém simplesmente digitou dez linhas pelo teclado. Ele não tem como saber que os dados vieram de um arquivo, então exibe apenas o número limpo.

Essa saída limpa faz diferença real quando combinamos o `<` com o `>` no mesmo comando. Vamos contar as linhas do log e salvar apenas o número em outro arquivo:

```
(base)      thallys@thallys-note:~/Documentos/Sonho/W2G$      wc      -l      <
logs/erro_critico.log > estatisticas.txt

(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat estatisticas.txt
10
```

O arquivo `estatisticas.txt` contém apenas o número `10`, sem o nome do arquivo de origem misturado. Se tivéssemos usado `wc -l logs/erro_critico.log > estatisticas.txt`, o arquivo conteria `10 logs/erro_critico.log`, o que seria mais difícil de processar automaticamente em um script.

O mesmo padrão funciona com o `sort`. Podemos ordenar o conteúdo de um arquivo e salvar o resultado em outro, tudo em uma única linha, sem nada aparecer na tela:

```
(base)      thallys@thallys-note:~/Documentos/Sonho/W2G$      sort      <
logs/erro_critico.log > logs/erros_ordenados.txt

(base)      thallys@thallys-note:~/Documentos/Sonho/W2G$      cat
logs/erros_ordenados.txt
Bom demais!
erro 1
eRro 1
erro 2
eRro 2
Erro 3
Erro 3
```

```
ERRO 4
eRRO 5
```

Nesse exemplo, os dados fluíram nos dois sentidos ao mesmo tempo: o `<` alimentou o `sort` pela entrada, o `sort` processou no meio do caminho, e o `>` salvou o resultado pela saída. Nenhuma informação passou pela tela.

Em resumo, o `<` não é algo que você usará em todos os comandos, mas ele se torna importante em duas situações: quando precisa de uma saída limpa sem metadados extras, e quando quer combinar entrada e saída no mesmo comando para fazer dados fluírem diretamente de arquivo para arquivo. Veremos esse padrão sendo usado nos scripts da Parte 4, especialmente quando precisamos capturar apenas um número para tomar decisões dentro do código.

1.3 Pipes

Na seção anterior, aprendemos a desviar os canais de comunicação para **arquivos**. Mas existe uma segunda forma de reconectar os canos, ainda mais poderosa: conectar a saída de um comando diretamente à entrada de **outro comando**, sem criar nenhum arquivo intermediário. Essa conexão é feita pelo operador `|`, chamado de **pipe** (tubo, em inglês).

Para entender por que isso é tão útil, vamos partir de um problema concreto dentro do nosso projeto. Imagine que queremos saber **quantas linhas** tem o nosso arquivo `logs/erro_critico.log`. Na seção anterior, aprendemos que o `wc -l` faz exatamente isso:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ wc -l
logs/erro_critico.log
10 logs/erro_critico.log
```

Funciona perfeitamente. Mas agora imagine uma pergunta diferente: **quantos arquivos .cfg existem na pasta configs/**? O `ls` lista os arquivos e o `wc -l` conta linhas, mas são dois comandos separados. Sem o pipe, precisaríamos de um arquivo temporário para conectá-los:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls configs/ > tmp.txt
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ wc -l tmp.txt
4 tmp.txt
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ rm tmp.txt
```

São três comandos para uma pergunta simples, além de criar e apagar um arquivo desnecessário. O pipe resolve isso conectando o stdout do `ls` diretamente ao stdin do `wc -l`, tudo em uma única linha:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls configs/ | wc -l
4
```

O que aconteceu aqui: o `ls configs/` gerou a listagem dos quatro arquivos `.cfg` e, em vez de imprimi-la na tela, enviou diretamente para o `wc -l`, que contou as linhas e exibiu o resultado. Nenhum arquivo temporário foi criado, tudo fluiu pela memória.

1.3.1 Encadeando múltiplos pipes

O poder real dos pipes aparece quando encadeamos **mais de dois** comandos em sequência, formando uma verdadeira linha de montagem onde cada comando faz uma etapa do processamento. Vamos explorar isso construindo cenários cada vez mais sofisticados dentro do nosso projeto.

Cenário 1: Descobrindo quais robôs acessaram o sistema

Imagine que temos um arquivo de log registrando cada acesso dos robôs ao servidor do laboratório. Vamos criá-lo:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ nano logs/acessos.log
```

```
roboZ
roboA
roboB
roboA
roboC
roboA
roboZ
```

Salve com `Ctrl+O` e saia com `Ctrl+X`. Queremos saber **quais robôs** acessaram, sem repetições. Já conhecemos o `sort` da seção anterior, que ordena linhas. Agora precisamos de um comando novo: o `uniq`.

O `uniq` remove linhas duplicadas de um texto, mas com uma particularidade importante: ele só detecta duplicatas que estejam **uma ao lado da outra**. Se as linhas repetidas estiverem espalhadas pelo arquivo, o `uniq` não as encontra. É por isso que **sempre usamos o `sort` antes do `uniq`**, para agrupar as linhas iguais primeiro:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ sort logs/acessos.log |
uniq
roboA
roboB
roboC
roboZ
```

O `sort` agrupou as linhas iguais em sequência, e o `uniq` eliminou as repetições. Agora sabemos quais robôs acessaram. Mas **quantas vezes** cada um acessou? A flag `-c` do `uniq` conta as ocorrências:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ sort logs/acessos.log |  
uniq -c  
3 roboA  
1 roboB  
1 roboC  
2 roboZ
```

Bom, mas o resultado está em ordem alfabética. Para ter um ranking real, do mais frequente para o menos, adicionamos mais um pipe com `sort -rn`, onde `-r` inverte a ordem e `-n` trata os números como valores numéricos em vez de texto:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ sort logs/acessos.log |  
uniq -c | sort -rn  
3 roboA  
2 roboZ  
1 roboC  
1 roboB
```

Nessa única linha, encadeamos **três pipes**. Cada comando fez uma etapa: o primeiro `sort` agrupou as linhas iguais, o `uniq -c` contou as ocorrências, e o segundo `sort -rn` organizou o ranking do maior para o menor. Essa é a filosofia Unix em ação: ferramentas pequenas que fazem uma coisa bem, combinadas para resolver problemas complexos.

Cenário 2: Extraíndo informações específicas com `cut`

Os pipes se tornam ainda mais úteis quando precisamos extrair **partes específicas** de cada linha. Imagine que o laboratório mantém um registro estruturado de todas as operações dos robôs. Vamos criar esse arquivo dentro do nosso projeto:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ nano logs/operacoes.log
```

```
08:00;roboA;patrulha;100%  
08:15;roboB;coleta;85%  
08:30;roboZ;mapeamento;92%  
09:00;roboA;recarga;45%  
09:15;roboC;patrulha;78%  
09:30;roboB;emergencia;12%  
10:00;roboZ;mapeamento;88%
```

Salve com `Ctrl+O` e saia com `Ctrl+X`. Cada linha segue o formato `horario;robo;tarefa;bateria`, com os campos separados por `;`. Agora queremos extrair apenas uma dessas informações, por exemplo, saber quais robôs aparecem no registro. Para isso, usamos o comando `cut`.

O **cut** recorta campos (colunas) de cada linha com base em um delimitador, que é o caractere usado para separar os campos. Ele precisa de duas flags para funcionar. A flag **-d** define **qual caractere** separa os campos. No nosso arquivo, o separador é o **;**. A flag **-f** define **qual coluna** queremos extrair, usando a posição numérica a partir de 1. O nome da flag vem de **field** (campo, em inglês), pois cada coluna separada pelo delimitador é tecnicamente chamada de campo. Combinando as duas flags, pedimos ao **cut** para recortar exatamente a coluna que nos interessa:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cut -d";" -f2  
logs/operacoes.log  
roboA  
roboB  
roboZ  
roboA  
roboC  
roboB  
roboZ
```

O **-d";"** disse ao **cut** que os campos são separados por ponto e vírgula, e o **-f2** selecionou o segundo campo de cada linha, que é o nome do robô. Podemos extrair outros campos alterando o número. Para ver apenas as tarefas (campo 3):

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cut -d";" -f3  
logs/operacoes.log  
patrulha  
coleta  
mapeamento  
recarga  
patrulha  
emergencia  
mapeamento
```

Também é possível extrair **mais de um campo** ao mesmo tempo, separando os números por vírgula. Para ver o horário (campo 1) e o nível de bateria (campo 4) juntos:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cut -d";" -f1,4  
logs/operacoes.log  
08:00;100%  
08:15;85%  
08:30;92%  
09:00;45%  
09:15;78%  
09:30;12%  
10:00;88%
```

Agora vem o poder real: combinar o **cut** com outros comandos usando pipes. Queremos saber **quais robôs** entraram em modo de emergência. O **grep** filtra as linhas que contêm a palavra **emergencia**, e o

`cut` extrai apenas o nome do robô dessas linhas:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ grep "emergencia"
logs/operacoes.log | cut -d";" -f2
roboB
```

Dois comandos, cada um fazendo sua parte, conectados por um pipe: o `grep` encontrou a linha relevante e o `cut` extraiu a informação que precisávamos. Podemos ir além e descobrir quais robôs operaram com bateria abaixo de 50%. Primeiro, o `grep` filtra as linhas, depois o `cut` extrai os nomes, e por fim o `sort` com `uniq` eliminam duplicatas:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ grep
"emergencia\\|recarga" logs/operacoes.log | cut -d";" -f2,4
roboA;45%
roboB;12%
```

Nesse exemplo, o `grep` usou o operador `\\|` para buscar linhas que contenham **uma palavra ou outra** (emergencia ou recarga), e o `cut` extraiu o nome do robô junto com o nível de bateria. O resultado nos mostra rapidamente quais robôs estavam em situação crítica e com quanto de bateria operavam naquele momento.

Cenário 3: Salvando resultados intermediários com `tee`

Em todos os exemplos anteriores, o resultado final apareceu na tela. Mas e se precisarmos **salvar** esse resultado em um arquivo **e ao mesmo tempo** continuar o processamento? Poderíamos usar o `>` no final, mas aí perderíamos a visualização na tela. É aqui que entra o `tee`.

O `tee` funciona como um conector em T de encanamento real: ele recebe os dados, envia uma cópia para um arquivo e passa os mesmos dados adiante pelo pipe. O fluxo não é interrompido, apenas duplicado. Vamos ver com um exemplo simples. Queremos listar os arquivos de configuração do projeto, salvar essa listagem em um arquivo **e ao mesmo tempo** contar quantos são:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ ls configs/ | tee
lista_configs.txt | wc -l
4
```

O resultado `4` apareceu na tela, mas o `tee` também salvou uma cópia da listagem no arquivo. Conferindo:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat lista_configs.txt
roboA.cfg
roboB.cfg
roboC.cfg
roboZ.cfg
```

Sem o `tee`, teríamos que escolher: ou víamos a contagem na tela, ou salvávamos a listagem no arquivo. Com ele, fizemos os dois ao mesmo tempo. O `ls` gerou a listagem, o `tee` duplicou o fluxo salvando uma cópia em `lista_configs.txt`, e o `wc -l` recebeu os mesmos dados e contou as linhas. O `tee` funciona como um conector em T de encanamento real: ele recebe os dados, envia uma cópia para um arquivo e passa os mesmos dados adiante pelo pipe. O fluxo não é interrompido, apenas duplicado.

Voltando ao nosso ranking de acessos dos robôs, imagine que além de ver o resultado na tela, queremos salvar a contagem em um arquivo para consultar depois. Podemos inserir o `tee` no meio do pipe, exatamente no ponto onde queremos capturar uma cópia dos dados:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ sort logs/acessos.log |  
uniq -c | tee contagem_robos.txt | sort -rn  
3 roboA  
2 roboZ  
1 roboC  
1 roboB
```

O ranking apareceu na tela normalmente, mas o `tee` também salvou uma cópia no arquivo `contagem_robos.txt`. Conferindo:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat contagem_robos.txt  
3 roboA  
1 roboB  
1 roboC  
2 roboZ
```

Observe um detalhe interessante: o conteúdo do arquivo está em **ordem alfabética**, não em ordem de ranking. Isso acontece porque o `tee` capturou os dados **antes** do último `sort -rn`, que veio depois dele no pipe. Cada comando na corrente processa os dados na ordem em que aparece, e o `tee` salvou o estado intermediário, exatamente como ele estava naquele ponto da linha de montagem.

Por fim, assim como o `>` tem o `>>`, o `tee` aceita a flag `-a` para adicionar ao arquivo em vez de sobrescrever:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ date | tee -a  
contagem_robos.txt  
qua 19 mar 2025 11:30:00 -03
```

Conferindo:

```
(base) thallys@thallys-note:~/Documentos/Sonho/W2G$ cat contagem_robos.txt  
3 roboA  
1 roboB  
1 roboC
```

2. O que é um Processo?

Antes de aprender a controlar processos, precisamos entender o que eles são e como surgem. Vamos começar do começo: o que acontece quando você digita um comando no terminal?

2.1 Do arquivo parado ao programa em execução

Imagine um livro de receitas guardado em uma prateleira. Enquanto está lá parado, ele não cozinha nada, é apenas uma coleção de instruções em papel. O mesmo vale para programas de computador.

Um **programa** é um arquivo armazenado no disco rígido contendo uma sequência de instruções que descrevem o que o computador deve fazer. O arquivo `/usr/bin/ls`, por exemplo, é um programa. Enquanto ninguém o executa, ele é tão inerte quanto qualquer outro arquivo, um PDF, uma imagem, não importa.

O que acontece quando você digita `ls` no terminal? O sistema operacional lê esse arquivo do disco e o carrega na **memória RAM**, a área de trabalho ativa do computador. É nesse momento que nasce um **processo**: um programa que saiu do disco, ganhou memória, recursos e vida própria. A receita saiu da prateleira e alguém começou a cozinhar.

Resumindo: programa é o arquivo parado no disco. Processo é esse programa em execução na memória.

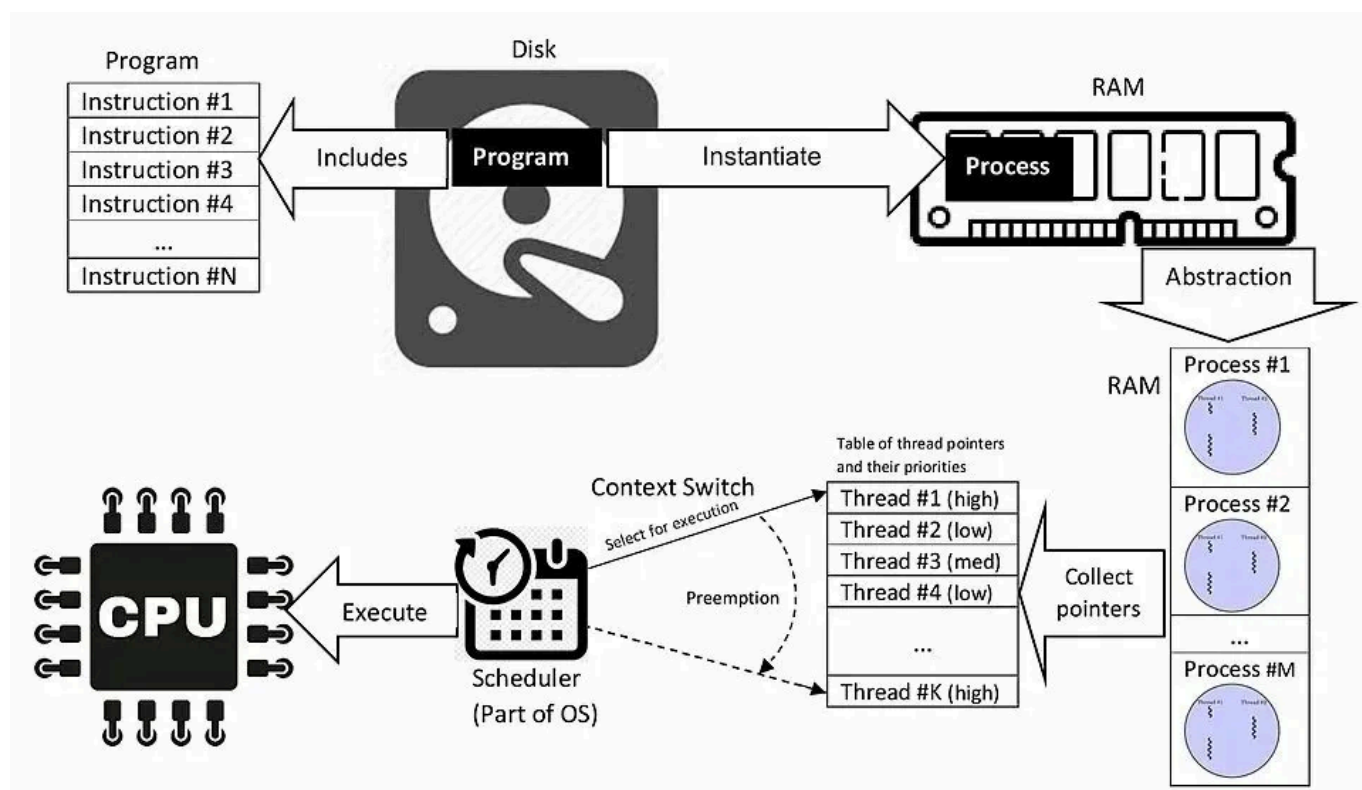


Figura 3: O caminho completo de um programa: do disco à CPU, passando pela criação de processos, threads e o scheduler do sistema operacional.

Observe a figura acima. No canto superior esquerdo está o **programa** no disco, com suas instruções empilhadas. A seta **Instantiate** mostra o momento em que ele é carregado na RAM e se torna um **processo**. No lado direito, a seta **Abstraction** mostra que o sistema pode ter muitos processos rodando ao mesmo tempo, cada um com suas próprias threads internas. Na parte inferior, o **Scheduler** coleta os ponteiros de todas as threads, organiza por prioridade e decide qual vai para a CPU executar. Esse é o ciclo completo que acontece por baixo dos panos cada vez que você digita um comando.

Um mesmo programa pode gerar vários processos ao mesmo tempo. Quando você abre três janelas do Firefox, cada janela é um processo independente, todas criadas a partir do mesmo arquivo `/usr/bin/firefox`, mas vivendo separadamente na memória.

2.2 O PID: o RG de cada processo

Para gerenciar todos esses processos ao mesmo tempo, o sistema operacional precisa identificar cada um deles de forma única. Para isso, ao criar um processo, o sistema atribui a ele um número exclusivo chamado **PID** (Process ID, Identificador de Processo).

O PID funciona como o RG de um cidadão: não existem dois processos com o mesmo PID ao mesmo tempo no sistema. O kernel mantém internamente uma **tabela de processos**, onde cada entrada registra o PID, o nome do programa e o estado atual de cada processo em execução, conforme ilustrado na figura abaixo.

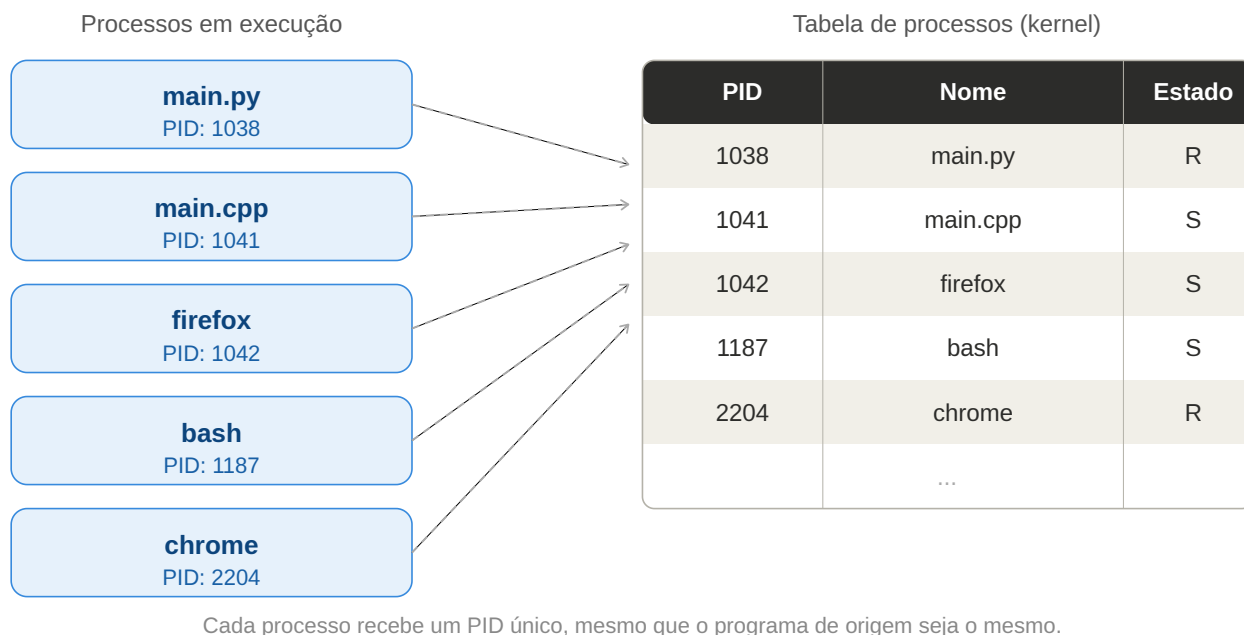


Figura 4: O kernel mantém uma tabela de processos onde cada processo recebe um PID único, independente do programa de origem.

É por meio desse número que o sistema identifica, monitora e encerra processos. Mais adiante, veremos comandos que usam o PID diretamente para interagir com processos específicos.

2.3 Threads: tarefas dentro de um processo

Até agora falamos de processos como unidades independentes. Pode acontecer de um único programa precisar fazer várias coisas ao mesmo tempo. É para isso que existem as **threads** ou linhas de execução.

Uma thread é uma tarefa que roda dentro de um processo. Ela funciona de forma independente, mas compartilha os mesmos recursos do processo principal, como a memória e os arquivos abertos. A melhor forma de entender isso é imaginar um apartamento.

O processo é um apartamento completo. Ele tem suas próprias paredes, sua cozinha e seu banheiro. O apartamento é totalmente isolado dos vizinhos. As **threads** são as pessoas que moram nesse apartamento. Cada pessoa realiza sua própria tarefa. Uma pessoa cozinha, outra limpa a sala e outra assiste televisão. Todas essas pessoas dividem o mesmo espaço e os mesmos móveis. Se uma pessoa quebrar a televisão, as outras também ficam sem assistir. Porém, o apartamento do vizinho continua intacto.

Observe a figura abaixo. Do lado esquerdo temos processos simples como **main.py**, **main.cpp** e **bash**. Esses programas possuem apenas uma thread cada. Eles são como apartamentos com um único morador. Do lado direito temos o **chrome**. Ele é um processo mais complexo com três threads rodando ao mesmo tempo. Cada thread faz uma tarefa diferente, mas todas dividem a mesma memória.

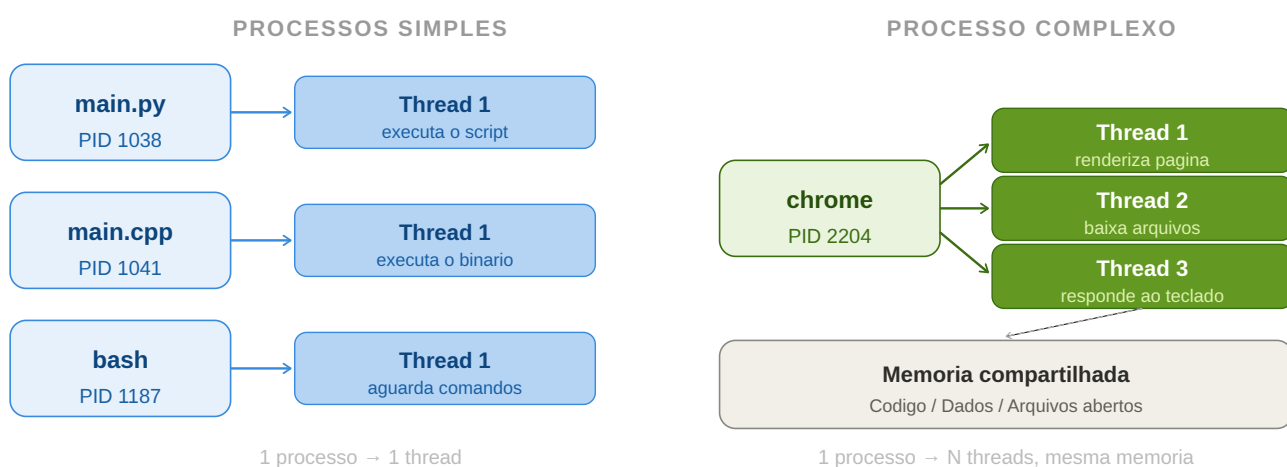


Figura 5: Processos simples possuem uma unica thread. Processos complexos criam multiplas threads que compartilham a mesma memoria.

O que cada thread tem de exclusivo é seu próprio contador de programa, seus registradores e sua pilha de execução. Esses itens guardam em que ponto do trabalho aquele fio está. O que todas as threads de um processo compartilham é o espaço de memória, as variáveis globais e os arquivos abertos. Isso significa que as threads são muito mais baratas de criar do que os processos, pois não precisam de memória própria. Por outro lado, isso também significa que uma thread com defeito pode corromper os dados das outras threads do mesmo processo.

Essa diferença fica bem clara na tabela abaixo.

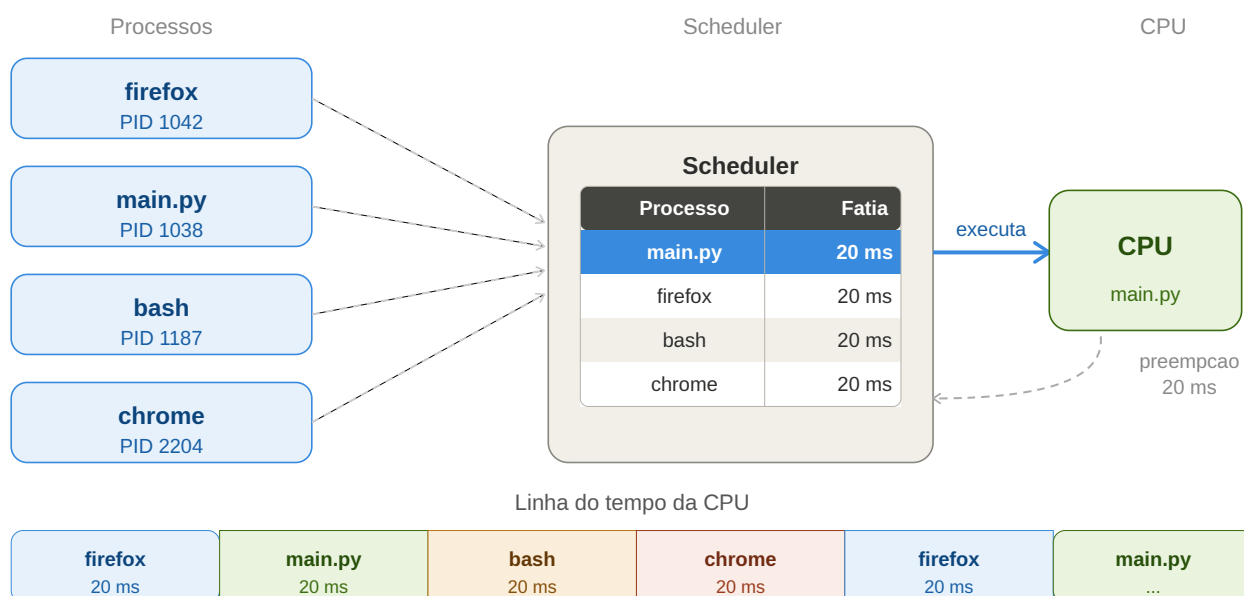
Característica	Processo	Thread
Memoria	Isolada dos outros processos	Compartilhada com as outras threads do mesmo processo
Custo de criacao	Alto, precisa de memoria propria	Baixo, aproveita a memoria do processo
Isolamento	Total, uma falha nao afeta outros processos	Parcial, uma thread com defeito pode afetar as outras do mesmo processo

Programas simples como **bash**, **cat** e **grep** normalmente rodam com uma única thread. Já programas complexos como navegadores web, servidores e o Docker utilizam múltiplas threads para realizar várias operações em paralelo. Nos comandos de inspeção que veremos a seguir, a letra l na coluna STAT indica que um processo possui múltiplas threads.

2.4 A ilusão da simultaneidade: multitarefa

Olhe para o seu computador agora. Provavelmente há um navegador aberto, um terminal, talvez um editor de texto e dezenas de programas funcionando **ao mesmo tempo**. Mas como isso é possível se o processador executa apenas uma instrução por vez?

A resposta é que isso é uma ilusão bem construída. O Linux é um sistema **multitarefa**. Em vez de executar um programa do começo ao fim para só então passar para o próximo, ele alterna entre as tarefas em altíssima velocidade. O sistema dá a cada thread uma pequena fatia de tempo no processador, na ordem de dezenas de milissegundos. Depois ele a pausa e passa a vez para a próxima. Esse ciclo de alternância se repete milhares de vezes por segundo.



A CPU executa um processo por vez em fatias de 20 ms. Para o usuário, parece simultâneo.

Figura 6: O scheduler divide o tempo da CPU em pequenas fatias de tempo e alterna entre as threads, criando a ilusão de simultaneidade.

Essa troca acontece tão rapidamente que o usuário tem a sensação de que tudo está rodando ao mesmo tempo. Na verdade, o processador está apenas sendo compartilhado em fatias minúsculas de tempo. Essa alternância é controlada por um componente do sistema operacional chamado **scheduler** (escalonador). Ele decide qual thread roda a seguir e por quanto tempo. Na figura acima você pode ver exatamente esse mecanismo. O scheduler mantém uma fila com todas as threads prontas para rodar, seleciona a próxima para executar na CPU e, quando o tempo acaba, realiza a **preempção**, que é o ato de interromper aquela thread e selecionar a próxima da fila.

Vale destacar que processos com apenas uma thread, como o **bash**, competem normalmente nessa fila. Já processos com múltiplas threads, como o **chrome**, colocam cada thread individualmente na fila. Isso

significa que um processo com três threads tem três oportunidades de ganhar tempo de CPU a cada ciclo, o que explica por que programas complexos respondem melhor ao usar múltiplas threads.

Na robótica esse mecanismo é fundamental. Em um robô em operação vários programas rodam em paralelo. Um controla a câmera, outro processa os dados do sensor LIDAR, outro envia comandos para os motores e mais um cuida da comunicação com a base. Se uma thread de um desses programas travar e consumir toda a CPU sem devolver a vez, o scheduler não consegue dar tempo suficiente às demais tarefas e os motores podem parar de responder. É por isso que saber inspecionar e encerrar processos é uma habilidade tão importante quanto saber programar.

2.5 Hierarquia de processos: pais e filhos

No Linux os processos não surgem do nada, pois todo processo é criado por outro programa que já estava rodando, sendo o criador chamado de processo **pai** e o novo processo conhecido como **filho**.

Quando você digita um comando como o **ls** no terminal, o **bash** (que é o processo do próprio terminal) entra em ação criando um novo processo filho exclusivamente para executar essa tarefa. Assim que o **ls** termina o seu trabalho, esse processo filho desaparece e o pai retoma o controle da situação.

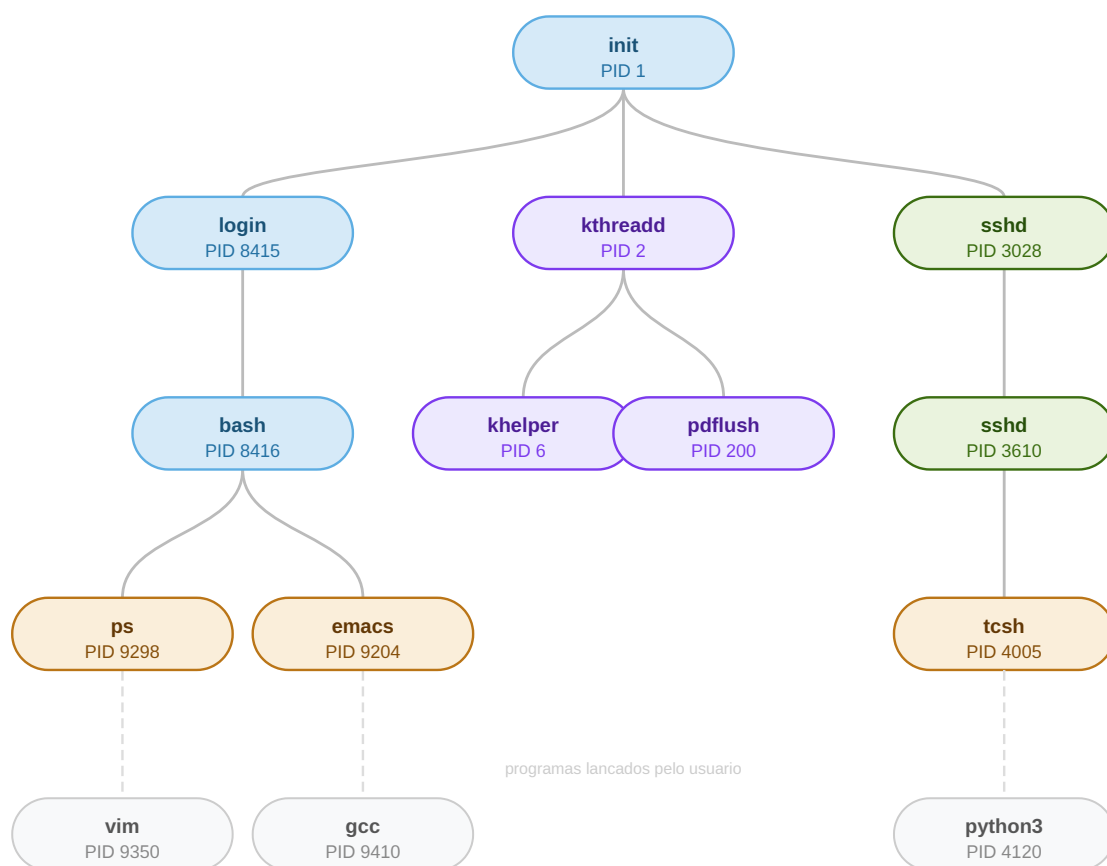


Figura 7: A árvore de processos do Linux. O **init** (PID 1) é o ancestral de todos os demais processos do sistema.

Toda essa cadeia de parentesco forma uma estrutura conhecida como **árvore de processos**, cujo topo é sempre ocupado por um processo principal de PID 1, o que nos obriga a conhecer os personagens mais importantes dessa hierarquia para compreendê-la de verdade.

O grande ancestral de todos é o **init** ou o **systemd**, sendo o primeiro o programa clássico e o segundo a versão adotada pelas distribuições mais modernas. Independentemente do nome, a função de ambos é atuar como o primeiro processo criado diretamente pelo kernel durante a inicialização da máquina, com a responsabilidade de ativar os serviços básicos, montar os discos e preparar todo o terreno para o uso do computador, fazendo com que absolutamente todos os outros processos derivem deles.

A partir desse ancestral nascem os processos de controle de acesso, como o **login**, que é o responsável por pedir seu usuário e senha quando você usa o computador fisicamente, e o **sshd**, que realiza esse mesmo trabalho para os acessos remotos ao permitir que você se conecte ao computador pela rede de forma segura. Sem esses processos de autenticação você simplesmente não conseguiria entrar no sistema.

Logo após a validação do seu acesso o sistema inicia o **bash**, que atua como o interpretador de comandos do seu terminal e serve como a interface principal de trabalho do usuário. É exatamente a partir do **bash** que nascem todos os comandos executados no dia a dia, incluindo as ferramentas de rede, os editores de texto e até mesmo os códigos do seu próprio projeto.

Compreender essa hierarquia ajuda a explicar um comportamento muito comum no dia a dia, pois quando você fecha uma janela do terminal, todos os programas que estavam rodando dentro dela são encerrados junto. O motivo para isso é simples, visto que o terminal atua como o processo pai e, ao ser interrompido, os seus processos filhos perdem a referência e acabam sendo encerrados em cascata pelo sistema.

2.6 Os estados de um processo

Assim como vimos que os processos nascem de uma hierarquia bem definida, é importante entender que, uma vez criados, eles não passam o tempo todo apenas executando instruções sem parar. Ao longo de sua existência, um processo transita continuamente entre diferentes **estados**, sempre dependendo exatamente do que está acontecendo com ele ou com o sistema naquele momento específico.

A melhor forma de compreender essa dinâmica é pensar em um funcionário de escritório trabalhando em sua mesa. Esse funcionário nem sempre está digitando furiosamente no teclado, pois, às vezes, ele fica aguardando um e-mail importante chegar; em outros momentos, ele é colocado em pausa para dar vez a uma reunião urgente e, eventualmente, ele pode até já ter terminado seu trabalho, mas ainda se encontra aguardando a dispensa formal do seu chefe. Um processo dentro do sistema operacional funciona de uma maneira muito parecida com esse funcionário.

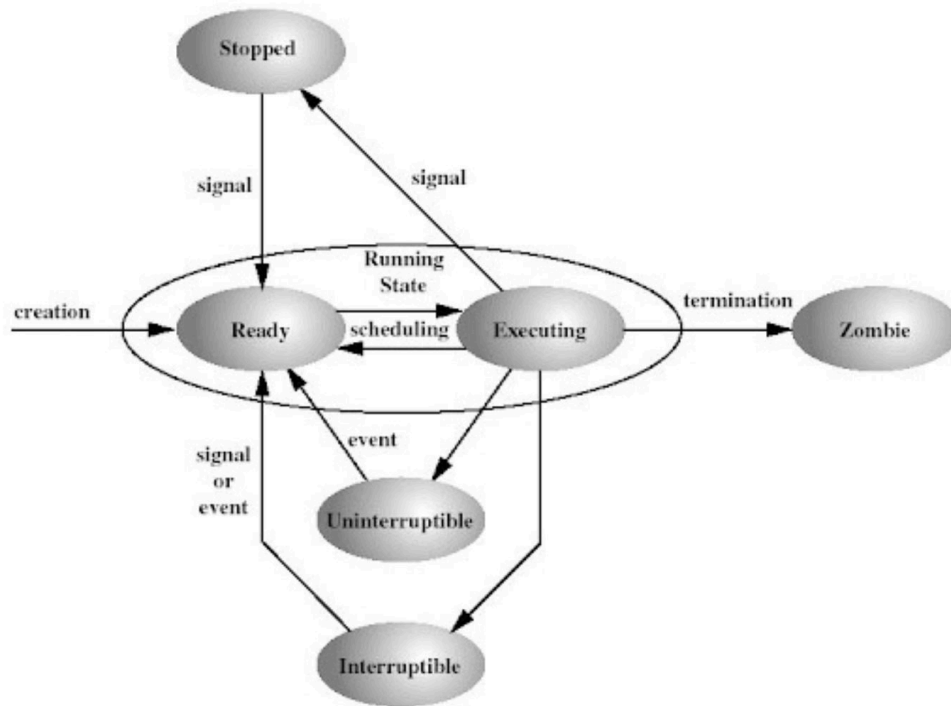


Figura 8: Diagrama de transição dos estados de um processo no sistema operacional

A figura acima ilustra exatamente esse fluxo. Note que o estado geral de execução (Running State) engloba tanto o momento em que o processo está de fato no processador (Executing) quanto o momento em que ele está na fila apenas aguardando a sua vez (Ready). Da mesma forma, os bloqueios se dividem entre os que podem ser interrompidos (Interruptible) e os críticos que não podem (Uninterruptible).

No dia a dia, quando utilizamos ferramentas de linha de comando (como ps ou top) para inspecionar o sistema, o kernel traduz esses conceitos teóricos do diagrama em códigos simples de uma letra. A tabela abaixo detalha essa classificação prática:

Estado	Nome	O que esta acontecendo
R	Running	O processo está no processador agora (Executing) ou na fila aguardando sua vez de executar (Ready).
S	Sleeping	Equivalente ao Interruptible. O processo está aguardando algum evento externo, como uma tecla pressionada, e pode ser acordado a qualquer momento.
D	Disk Sleep	Equivalente ao Uninterruptible. Igual ao estado Sleeping, mas aguardando operações críticas de disco ou rede que não podem ser interrompidas.
T	Stopped	O processo foi pausado manualmente pelo usuário e continua na memória, porém sem executar nenhuma ação.
Z	Zombie	O processo terminou seu trabalho, mas o processo pai ainda não coletou seu resultado, virando um registro fantasma.
I	Idle	Indica um processo interno do kernel que se encontra completamente ocioso.

O ciclo de vida mais comum de um programa começa com ele nascendo no estado **Running**, de onde frequentemente transita para o estado **Sleeping** enquanto espera o usuário digitar algo ou um arquivo ser lido, voltando imediatamente para o estado de execução quando esse evento externo finalmente acontece.

Dentre todos esses cenários, o estado **Zombie** merece uma atenção especial da nossa parte. Quando um processo filho termina a sua tarefa, ele não desaparece imediatamente da memória, mas sim entra nesse estado zumbi, aguardando que o seu processo pai leia o seu código de saída — exatamente como o funcionário que terminou a tarefa e espera o chefe liberar sua saída formal. Se o processo pai for mal programado e nunca fizer essa leitura, o processo filho fica preso para sempre como um fantasma na tabela do sistema. Em situações normais, isso se resolve rapidamente, mas, em sistemas complexos, um acúmulo de zumbis costuma ser um sintoma claro de defeito no processo pai.

Agora que entendemos perfeitamente o que são os processos, como eles nascem em hierarquia, como se subdividem em threads e em quais estados eles podem estar durante a sua vida útil, nós finalmente temos toda a base conceitual necessária para utilizar os comandos de inspeção e controle na prática.

3. Inspeção

Com toda a base teórica consolidada chegou a hora de colocar a mão na massa. Nós já sabemos que existem centenas de processos e múltiplas threads rodando escondidos no sistema operacional neste exato momento. O problema é que toda essa estrutura é totalmente invisível para o usuário comum. Então é fundamental conseguir enxergar exatamente o que está acontecendo nos bastidores, mas nós precisamos saber quais programas estão consumindo muita memória, quais estão travados em um estado problemático e quem gerou quem dentro daquela árvore de execução. É exatamente para isso que servem os comandos de inspeção, pois eles funcionam como uma verdadeira janela de raio x para dentro do cérebro do computador e garantem que você tenha o controle total sobre a máquina.

3.1 Fotografia Instantânea do Sistema: **ps**

O comando **ps** (**p**rocess **s**tatus) tira uma "fotografia" dos processos em execução naquele exato momento. Quando executado sem argumentos, ele exibe apenas os processos iniciados no **seu terminal**, resultando em uma visualização restrita ao contexto atual:

```
(base) thallys@thallys-note:~/Repositories/Sonho$ ps
  PID TTY          TIME CMD
 18059 pts/0        00:00:01 zsh
 18135 pts/0        00:00:00 zsh
 18137 pts/0        00:00:00 zsh
 18139 pts/0        00:00:00 zsh
 18144 pts/0        00:00:00 gitstatusd-linu
 35938 pts/0        00:00:00 bash
 35974 pts/0        00:00:00 ps
```

Analisando o exemplo acima, vemos que há um pequeno grupo de processos atrelados a esta sessão do terminal. Entre eles estão algumas instâncias do seu shell (**zsh** e **bash**), um processo de auxílio do git (**gitstatusd-linu**) e o próprio comando **ps** (que estava executando no momento da "fotografia").

Para ver **todos** os processos do sistema de forma detalhada, usamos a combinação clássica de *flags* **aux**. Essa combinação mostra os processos de todos os usuários (**all users**), em um formato detalhado e amigável (**user-friendly**), incluindo aqueles que rodam em segundo plano sem estarem presos a um terminal (**x**).

Como a saída desse comando costuma ser imensa (centenas de processos rodando no kernel, serviços do sistema, aplicativos como o Google Chrome, contêineres do Docker, etc.), veja abaixo um recorte representativo do que é exibido:

```
(base) thallys@thallys-note:~/Repositories/Sonho$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1   0.0  0.0  167320 12752 ?        Ss   07:42   0:01 /sbin/init sp
root        2154   0.0  0.2 3324064 90272 ?        Ssl  07:42   0:03 /usr/bin/dockerd
999          2597   0.4  0.5 3723720 190204 ?        Ssl  07:42   0:46 mongod -
-noau
thallys     18059   0.0  0.0  17428 10964 pts/0    Ss   09:51   0:01 zsh
thallys     20476   2.5  2.4 51720412 795032 ?        SLl  09:55   0:59 /opt/google/chrome
thallys     36671   0.0  0.0  13076   3880 pts/0    R+   10:34   0:00 ps aux
...
```

Cada coluna da saída anterior revela uma informação crucial sobre o comportamento do processo. Note que incluímos também as colunas **START** e **TIME** que apareceram no nosso exemplo:

Coluna	Significado
USER	O usuário dono do processo.
PID	ID único do processo. Funciona como o "RG" do programa, que usaremos mais tarde para inspecioná-lo ou encerrá-lo.
%CPU	Percentual de processador que o processo está consumindo no exato momento da consulta.
%MEM	Percentual de memória RAM física que o processo está consumindo.
VSZ	Quantidade total de memória virtual que o processo requisitou ao sistema.
RSS	Quantidade de memória física real que o processo está ocupando ativamente na sua RAM.
TTY	O terminal de texto associado ao processo (um ponto de interrogação ? indica que ele roda em segundo plano, sem terminal).
STAT	Estado atual do processo (veja as tabelas abaixo para todos os detalhes).
START	A hora exata em que o processo foi iniciado.
TIME	O tempo total de processador (CPU) que o processo acumulou desde o seu início.
COMMAND	O comando exato que originou o processo.

A coluna **STAT** (Status) merece uma atenção especial, pois ela permite "ler a mente" do kernel e entender o que o processo está fazendo naquele instante. Como você pôde notar no recorte da listagem geral, essa coluna costuma exibir mais de uma letra combinada (como **Ss**, **Ssl** ou **R+**).

Isso acontece porque a primeira letra indica o **Estado Principal** do processo, enquanto as letras seguintes funcionam como **Modificadores** que trazem detalhes extras sobre a forma como ele está sendo executado:

1. Estados Principais (Primeira Letra)

Estado	Nome	Significado
R	Running	Rodando ativamente no processador ou pronto na fila aguardando sua vez de rodar.
S	Interruptible Sleep	Dormindo e ocioso, aguardando um evento (como o usuário digitar no teclado ou um timer).
D	Uninterruptible Sleep	Aguardando operações críticas de disco ou rede (não pode ser interrompido até concluir).
T	Stopped	Parado após ser suspenso manualmente pelo usuário (usando Ctrl+Z , por exemplo).
Z	Zombie	Encerrado, porém o processo pai ainda não leu o seu código de saída.
I	Idle	Processo interno do núcleo (kernel) que está completamente ocioso.

2. Modificadores (Letras Adicionais)

Modificador	Significado
s	O processo é líder de sessão, geralmente o primeiro processo de um terminal.
l	O processo possui múltiplas <i>threads</i> , ou seja, executa várias tarefas internas simultaneamente.
+	O processo está rodando em primeiro plano (<i>foreground</i>) no terminal atual.
<	O processo possui alta prioridade, ganhando preferência no uso do processador.
N	O processo possui baixa prioridade, cedendo espaço para processos mais importantes.

Aplicando a teoria ao nosso exemplo prático

Agora que conhecemos o significado de cada coluna e os códigos da coluna **STAT**, podemos usar esse conhecimento para ler a "história" completa que o **ps aux** conta sobre cada processo. Veja o que as duas primeiras linhas da listagem nos dizem:

```
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0  0.0 167320 12752 ?        Ss     07:42    0:01
/sbin/init
root         2  0.0  0.0     0     0  ?        S      07:42    0:00
[kthreadd]
```

- O processo **/sbin/init** com PID 1 é o ancestral de todos os processos do sistema, aquele que vimos no topo da árvore de hierarquia. Ele roda com privilégios máximos de **root** e, no instante da

fotografia, não consumia nada de %CPU nem de %MEM. Mesmo assim, mantinha alocados cerca de 167 MB de memória virtual (VSZ) e 12 MB de memória física real (RSS), recursos reservados para coordenar o sistema. O ? na coluna TTY confirma que ele roda silenciosamente em segundo plano, sem nenhum terminal anexado. Seu estado Ss revela exatamente o que aprendemos: o S maiúsculo indica que ele está dormindo (*Interruptible Sleep*), aguardando eventos do sistema, enquanto o s minúsculo confirma que ele é o líder de sessão que gerencia os demais serviços. Ele foi iniciado durante o boot às 07:42 e acumulou apenas 1 segundo de tempo total de CPU desde então.

- O [kthreadd] com PID 2 também pertence ao root, mas apresenta 0 em VSZ e em RSS. Essa ausência completa de alocação de memória, combinada com o nome escrito entre colchetes, é a assinatura visual de uma thread interna do próprio kernel. Threads do kernel não precisam de memória de usuário porque vivem diretamente no espaço do núcleo. Seu estado exibe apenas um S isolado, sem nenhum modificador adicional, indicando que ele simplesmente repousa ocioso aguardando ordens do kernel, sem ser líder de sessão e sem estar atrelado a nenhum terminal.

Agora olhe para as linhas mais interessantes do nosso recorte:

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	2154	0.0	0.2	3324064	90272	?	Ss1	07:42	0:03	/usr/bin/dockerd
thallys	20476	2.5	2.4	51720412	795032	?	SL1	09:55	0:59	/opt/google/chrome
thallys	36671	0.0	0.0	13076	3880	pts/0	R+	10:34	0:00	ps aux

- O dockerd apresenta o estado Ss1: está dormindo (S), é líder de sessão (s) e possui múltiplas threads (1). Isso faz todo sentido para um daemon de contêineres, que precisa gerenciar várias operações em paralelo ao mesmo tempo que aguarda conexões. Já o Chrome mostra SL1, onde o L indica que ele usa memória bloqueada em RAM (páginas que o sistema operacional não pode mover para o disco), além das múltiplas threads esperadas de um navegador moderno. Por fim, o próprio ps aux aparece com R+: está em estado Running porque está executando ativamente no momento da fotografia, e o + confirma que roda em primeiro plano no terminal atual, exatamente o que esperamos de um comando que acabamos de digitar.

Essa leitura combinada das colunas é o que transforma o ps aux de uma lista confusa de números em um diagnóstico preciso do que está acontecendo dentro do sistema. Na maioria das vezes, porém, não precisamos analisar todos os processos de uma vez. O que realmente queremos é encontrar um processo específico. Na prática, uma maneira mais interessante é utilizar ps aux | grep nome, que filtra processos pelo nome do programa. Em vez de percorrer centenas de linhas manualmente, o grep filtra pelo nome, deixando passar apenas as linhas que contêm o termo buscado:

```
(base) thallys@thallys-note:~/Repositories/Sonho$ ps aux | grep python3
root          882   0.0   0.0  41216 21756 ?        Ss   07:42   0:00
/usr/bin/python3 /usr/bin/networkd-dispatcher
thallys      55109   0.0   0.0   9240  2656 pts/0    S+   11:24   0:00 grep --
color=auto python3
```

O resultado mostra dois processos. O primeiro é um serviço do sistema rodando com privilégios de **root**, estado **Ss**, sem terminal associado (?), iniciado durante o boot. O segundo é o próprio **grep** que acabamos de executar, o que é completamente normal e sempre aparece. Se houvesse um script travado consumindo CPU, ele apareceria aqui com estado **R+** e um percentual alto em **%CPU**, e usaríamos o PID exibido para encerrá-lo, como veremos na próxima seção.

3.2 Hierarquia visual: **pstree**

O **pstree** exibe a relação de parentesco entre os processos como uma árvore visual, deixando claro quem criou quem. Sua sintaxe básica é:

```
pstree [modificadores]
```

Modificador	Efeito
-p	Exibe o PID de cada processo ao lado do nome
-a	Mostra os argumentos de linha de comando usados para iniciar cada processo
-n	Ordena os processos pelo PID em vez de ordenar por nome
-c	Desativa a compactação de ramos idênticos, exibindo cada instância separadamente
-h	Destaca o processo atual e todos os seus ancestrais na árvore

Rodando apenas **pstree**, sem flags, o sistema já compacta automaticamente os ramos repetidos usando a notação **N*[nome]**, onde **N** é a quantidade de instâncias idênticas. Isso acontece porque muitos processos criam dezenas de threads internas com o mesmo nome, e exibir cada uma em uma linha separada tornaria a saída ilegível. Em vez disso, **19*[{containerd}]** significa que o **containerd** possui 19 threads internas rodando, e **2*[docker-proxy]—7*[{docker-proxy}]** significa que há 2 instâncias do **docker-proxy**, cada uma com 7 threads próprias:

```
(base) thallys@thallys-note:~/Repositories/Sonho$ pstree
systemd─┬─containerd─19*[{containerd}]
          │
          └─dockerd─2*[docker-proxy]─7*[{docker-proxy}]
                  └─27*[{dockerd}]
          │
          └─systemd─┬─gnome-terminal─┬─zsh─bash─pstree
                        │               └─3*[{gnome-terminal-}]
                        └─4*[zsh]
          ...
```

Vale notar que os nomes entre chaves, como **{dockerd}** e **{gnome-terminal-}**, representam threads internas do processo, não processos filhos separados. O **dockerd**, por exemplo, mantém 27 threads rodando simultaneamente para gerenciar contêineres, conexões e operações de rede em paralelo.

Adicionando **-p** para ver os PIDs, a hierarquia fica ainda mais informativa:

```
(base) thallys@thallys-note:~/Repositories/Sonho$ pstree -p
systemd(1)─┬─dockerd(2154)─┬─docker-proxy(2734)─┬─{docker-proxy}(2735)
              │               └─27*[{dockerd}]
              └─systemd(3070)─┬─gnome-terminal-
(18037)─┬─zsh(18059)─┬─bash(35938)─┬─pstree(...)
              │               │               └─3*[{gnome-terminal-}]
              │               └─4*[{zsh}]
              └─...
```

Os números entre parênteses são os PIDs de cada processo, aquele identificador único que estudamos anteriormente. Assim, `systemd(1)` é o processo raiz com PID 1, `dockerd(2154)` é o daemon do Docker com PID 2154, e assim por diante. Essa informação é especialmente útil quando você precisa encerrar um processo específico, pois é o PID que os comandos de controle utilizam, como veremos mais adiante.

O caminho `gnome-terminal-(18037) → zsh(18059) → bash(35938) → pstree(...)` conta a história completa da sessão atual: o terminal com PID 18037 abriu um shell `zsh` com PID 18059, que por sua vez abriu um `bash` com PID 35938, que finalmente executou o `pstree` que estamos vendo agora. Cada processo filho recebeu um PID maior que o pai, o que reflete a ordem cronológica em que foram criados. Essa cadeia explica por que fechar a janela do terminal encerra todos os processos dentro dela: o pai é encerrado e leva os filhos em cascata.

3.3 Monitor em Tempo Real: `top` e `htop`

O `ps` e o `pstree` tiram uma fotografia do sistema em um instante específico. Quando você precisa **observar o sistema ao longo do tempo**, acompanhar o que está consumindo CPU ou memória agora e reagir a mudanças em tempo real, o comando certo é o `top`:

```
(base) thallys@thallys-note:~/Repositories/Sonho/W2G$ top
```

O `top` abre um painel interativo que se atualiza automaticamente a cada 3 segundos. A tela é dividida em duas partes: um **cabeçalho** com o resumo geral do sistema, e uma **lista de processos** ordenada por consumo de CPU.

 *Figura 9: Saída do comando `top`. No cabeçalho, *load average de 0,47* indica sistema tranquilo. Das 511 tarefas, apenas 1 está em execução ativa e 510 estão dormindo.*

O cabeçalho condensa o estado atual da máquina em quatro linhas:

```
top - 15:21:25 up 49 min,  1 user,  load average: 0,47, 0,42, 0,64
Tarefas: 511 total,   1 em exec.,  510 dormindo,   0 parado,   0 zumbi
%CPU(s):   0,7 us,   0,4 sy,   0,0 ni, 98,9 id,   0,1 wa,   0,0 hi,   0,0 si,
0,0 st
MB mem :   31779,0 total,   16564,1 livre,   7322,8 usados,   7892,1
buff/cache
MB swap:   7816,0 total,   7816,0 livre,   0,0 usados,  22881,1 mem
dispon.
```

A primeira linha mostra o horário atual, há quanto tempo o sistema está ligado (49 minutos neste caso), quantos usuários estão conectados e a **carga média** nos últimos 1, 5 e 15 minutos. Os valores 0,47, 0,42 e 0,64 indicam um sistema tranquilo. Como a máquina possui 20 núcleos, qualquer valor abaixo de 20,0 significa que ainda há capacidade ociosa disponível. A segunda linha mostra o total de tarefas por estado: das 511, apenas 1 está executando ativamente na CPU naquele instante e as outras 510 estão dormindo, aguardando algum evento. A terceira linha detalha como a CPU está distribuída: **us** são os processos do usuário, **sy** o kernel, **id** o tempo ocioso e **wa** o tempo aguardando disco. Com 98,9% de **id**, a CPU está praticamente ociosa nesse momento. As duas últimas linhas mostram o uso de memória RAM e swap: dos 31 GB totais, apenas 7,3 GB estão em uso direto e 7,9 GB estão em cache. O swap completamente livre confirma que o sistema não está sob pressão de memória.

Dentro do **top** você pode interagir diretamente pelo teclado sem precisar sair do painel:

Tecla	Ação
q	Encerra o top e retorna ao terminal
P	Ordena a lista por uso de CPU, colocando os processos mais pesados no topo
M	Ordena a lista por uso de memória RAM
k	Solicita um PID e envia um sinal de encerramento ao processo
r	Solicita um PID e permite alterar sua prioridade de execução (<i>renice</i>)
1	Alterna entre a visão consolidada de CPU e o detalhamento por núcleo individualmente
h	Abre o menu de ajuda completo com todos os atalhos disponíveis

O **top** cumpre bem seu papel, mas sua interface em texto puro pode ser difícil de ler quando o sistema está sob carga e dezenas de processos aparecem na tela ao mesmo tempo. O **htop** resolve isso com uma interface visual mais rica. Se não estiver instalado:

```
sudo apt install htop
```

Para utilizar:

```
(base) thallys@thallys-note:~/Repositories/Sonho$ htop
```

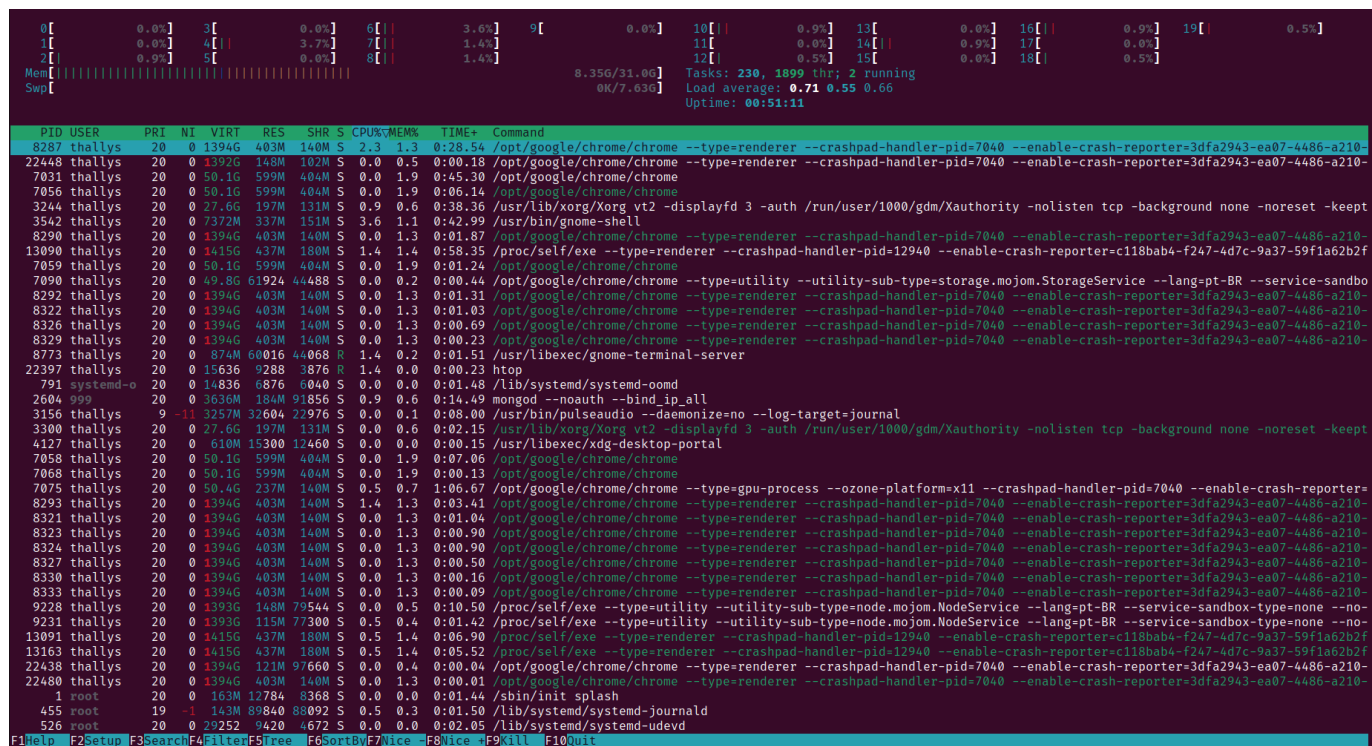


Figura 10: Saída do **htop**. No topo, barras individuais para cada um dos 20 núcleos da CPU, seguidas do uso de memória (8.35G de 31.0G) e swap. A lista mostra 230 processos com 1899 threads no total.

A navegação no **htop** é feita pelas teclas de função exibidas na barra inferior da tela. Você pode usar as setas do teclado para mover o cursor pela lista de processos e, com o processo desejado destacado, acionar qualquer uma das teclas abaixo:

Tecla	Ação
F1	Abre a tela de ajuda com todos os atalhos disponíveis.
F2	Abre o menu de configuração para personalizar colunas, cores e opções de exibição.
F3	Busca um processo pelo nome, destacando o primeiro resultado encontrado.
F4	Filtra a lista, ocultando todos os processos que não correspondem ao termo buscado.
F5	Alterna para a visão em árvore, mostrando a hierarquia pai-filho entre os processos.
F6	Escolhe a coluna de ordenação da lista
F7	Aumenta a prioridade do processo selecionado (<i>nice</i> mais alto).
F8	Diminui a prioridade do processo selecionado (<i>nice</i> mais baixo).
F9	Abre o menu de sinais para encerrar o processo selecionado, sem precisar digitar o PID.
F10	Sai do htop e retorna ao terminal.

Para o dia a dia, o fluxo mais comum é abrir o **htop**, pressionar **F3**, digitar o nome do processo que está pesando, pressionar **Enter** para localizá-lo na lista e usar **F9** para encerrá-lo.

A principal diferença prática é que o **top** está disponível em qualquer sistema Linux, incluindo servidores remotos acessados via SSH sem interface gráfica. O **htop** é mais agradável no dia a dia, mas pode não estar presente em ambientes minimalistas. Vale conhecer os dois.

3.4 Foreground, Background e Jobs

Por padrão, quando você executa qualquer comando no terminal, ele roda em **foreground** (primeiro plano): o terminal fica travado, mostrando a saída do comando e bloqueando qualquer nova entrada até que ele termine. Para comandos rápidos como `ls` ou `grep` isso não é problema. Mas e quando você precisa iniciar um processo que demora minutos ou horas, como um servidor web, um treinamento de modelo ou uma compilação pesada?

É aí que entra o conceito de **background** (segundo plano). Um processo rodando em background continua executando normalmente, mas devolve o controle do terminal para você imediatamente. Pense como abrir um programa pelo menu do sistema operacional: o programa abre e continua rodando, mas você ainda pode usar o computador normalmente.

Para praticar os exemplos a seguir, vamos usar um processo simples que fica rodando indefinidamente, simulando um script longo. Crie o arquivo `loop.py` com o seguinte conteúdo:

```
import time

while True:
    time.sleep(2)
```

Iniciando um processo diretamente em background

Para enviar um comando direto para o background, basta adicionar `&` ao final:

```
~/Doc/Sonho/W2G python3 scripts/loop.py &
[1] 117771
```

O shell responde com duas informações: `[1]` é o número do **job**, que é o identificador interno do processo dentro daquele terminal, e `117771` é o PID do processo. O terminal volta ao prompt imediatamente e o script continua rodando em silêncio.

Consultando o que está rodando em background

O comando `jobs` lista todos os processos em background do terminal atual:

```
~/Doc/Sonho/W2G jobs
[1] + running python3 scripts/loop.py
```

Suspendendo um processo que já está em foreground

Se você esqueceu o `&` e o terminal já está travado, não precisa cancelar o processo. Primeiro traga-o para o foreground com `fg`, depois use `Ctrl+Z` para suspendê-lo:

```
~/Doc/Sonho/W2G fg %1
[1] + 117771 running python3 scripts/loop.py
^Z
[1] + 117771 suspended python3 scripts/loop.py
```

Ctrl+Z apenas **pausa** o processo, ele não continua executando em segundo plano. Para retomá-lo em background, use **bg**:

```
~/Doc/S/W2G bg
[1] + 117771 continued python3 scripts/loop.py

~/Doc/Sonho/W2G jobs
[1] + running python3 scripts/loop.py
```

Encerrando um processo em background

Processos em background não respondem a **Ctrl+C** nem a **Ctrl+Z**, pois esses atalhos só funcionam no processo que está em foreground. Para encerrar, traga o processo de volta com **fg** e então use **Ctrl+Z** para suspendê-lo:

```
~/Doc/S/W2G fg %1
[1] - 117771 running python3 scripts/loop.py
^Z
[1] + 117771 suspended python3 scripts/loop.py
```

Caso prefira encerrar sem trazer para o foreground, use **kill** diretamente pelo número do job ou pelo PID encontrado com **ps aux | grep loop.py**:

```
# Pelo número do job
~/Doc/Sonho/W2G kill %1

# Ou pelo PID
~/Doc/Sonho/W2G kill 117771
```

4. Encerrando Processos

Saber encerrar processos é uma habilidade essencial. Um script travado consumindo 100% de CPU, um servidor que parou de responder ou um programa que simplesmente não fecha — todas essas situações exigem o **kill**.

O **kill** envia um **signal** ao processo identificado pelo PID. Por padrão, sem nenhuma flag, ele envia o sinal **SIGTERM**, que pede gentilmente ao processo para encerrar, dando tempo para ele salvar dados e fechar conexões:

```
kill 2450
```

Se o processo ignorar o **SIGTERM** e continuar rodando, use **-9** para forçar o encerramento imediato:

```
kill -9 2450
```

Os principais sinais que o **kill** pode enviar são:

Sinal	Número	Comportamento
SIGTERM	15	Encerramento gentil. O processo pode salvar dados antes de sair. É o padrão.
SIGKILL	9	Morte imediata. Sem chance de limpeza. Último recurso.
SIGINT	2	Equivalente ao Ctrl+C .
SIGSTOP	19	Suspende o processo, equivalente ao Ctrl+Z .

Nunca comece pelo **kill -9.** O SIGKILL não permite que o processo salve dados, limpe arquivos temporários ou feche conexões abertas. Sempre tente **kill PID** primeiro e aguarde alguns segundos. Só use **-9** se o processo realmente não responder.

O fluxo completo na prática é sempre o mesmo: encontrar o PID, tentar o encerramento gentil e confirmar que o processo foi encerrado. Primeiro, localizamos o processo pelo nome:

```
~/Doc/Sonho/W2G ps aux | grep loop.py
thallys      37824  0.0  0.0  18604  9280 pts/2    SN   10:04   0:00 python3
scripts/loop.py
thallys      37908  0.0  0.0   9240   2648 pts/2    S+   10:04   0:00 grep --
color=auto loop.py
```

Com o PID em mãos, enviamos o sinal de encerramento gentil:

```
~/Doc/Sonho/W2G kill 37824
[1] + 37824 terminated python3 scripts/loop.py
```

Se o processo ignorar o **SIGTERM** e continuar em execução, usamos **-9** para forçar o encerramento imediato. Por fim, rodamos o **ps aux | grep loop.py** novamente para confirmar. Quando apenas a linha do próprio **grep** aparece no resultado, sem nenhuma linha com **loop.py** como processo, o encerramento foi bem-sucedido:

```
~/Doc/Sonho/W2G ps aux | grep loop.py
thallys      37986  0.0  0.0   9240   2648 pts/2    S+   10:04   0:00 grep --
```

```
color=auto loop.py
```